Busca

Professor: Elton Sarmanho[1]
E-mail: eltonss@ufpa.br

[1]Faculdade de Sistemas de Informação - UFPA/CUTINS

3 de março de 2023

Roteiro

Introdução
- Objetivos
- Conceitos Gerais

Busca Sequencial
- Conceitos Gerais

Roteiro

Busca Binária

Conceitos Gerais

Trabalho 1 - Busca

- Esta aula apresenta conceitos gerais de Pesquisas ou Buscas em uma estrutura de dados. Ao final, você deverá compreender os seguintes tópicos:
  - Compreender o processo de Busca ou Pesquisa;
  - Algoritmos de Busca;
  - Aplicações Reais.

A importância em estudar busca

- Busca é uma tarefa muito comum em problemas da área da computação
- Vários métodos e estruturas de dados podem ser empregados para se fazer busca:
  - Listas
  - Pilhas
  - Grafos
  - Árvores
- Processo de ordenação de dados podem tornar o processo de busca mais eficiente

A importância em estudar busca

O problema da busca:

- *Dado um conjunto de elementos, onde cada um é identificado por uma chave, o objetivo da busca é localizar, nesse conjunto, o elemento que corresponde a uma chave específica*

Nomeclatura

- **Tabela ou Arquivo**: Qualquer estrutura de dados usada para armazenamento interno e organização dos dados:
  - é um conjunto de elementos, chamados **registros**
  - Existe uma **chave** associada a cada registro, usada para diferenciar os registros entre si

Nomeclatura

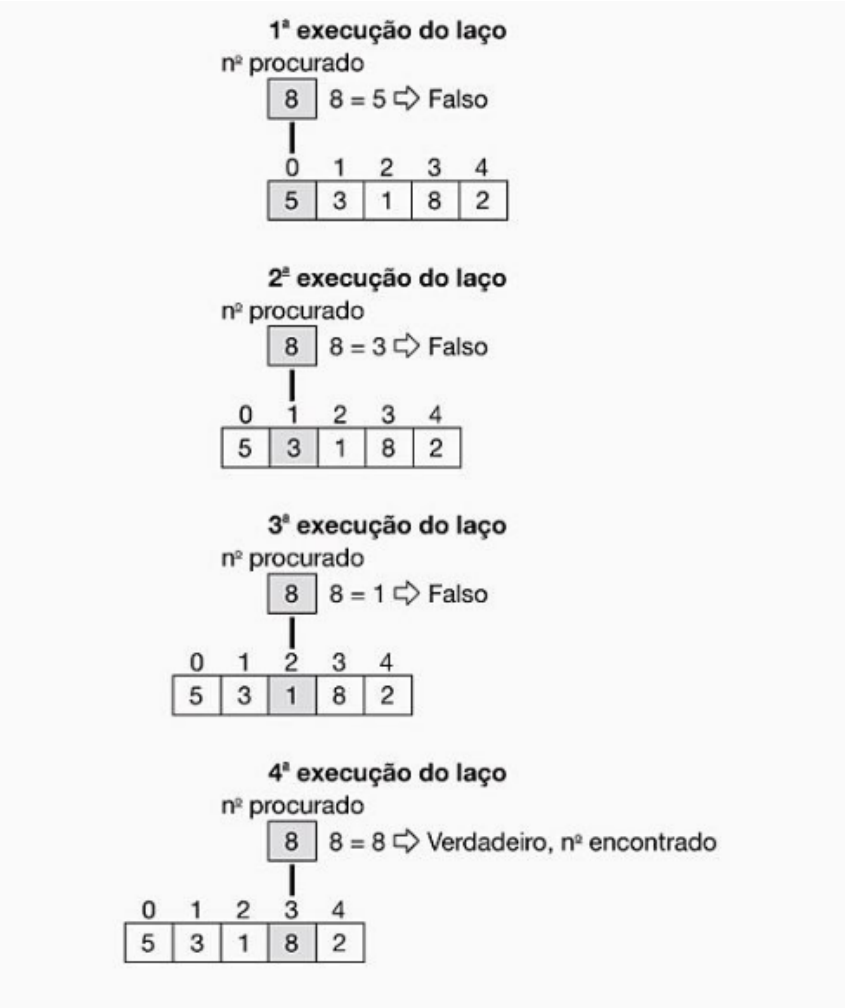
- **Tabela ou Arquivo** pode ser:
  - Um vetor
  - Uma Lista
  - árvore e etc
- **Tabela ou Arquivo** pode ficar:
  - Totalmente na memória (**busca interna**)
  - Totalmente no armazenamento auxiliar (busca externa)
  - Híbrida

Tipos de Busca

- As técnicas de busca abordadas nesta serão:
  - Busca Sequencial
  - Busca Binária
- O objetivo é encontrar um dado registro com o **menor custo**
- Cada técnica possui **vantagens** e **desvantagens**.

- A busca sequencial é a forma mais simples de busca
- Normalmente aplicada em listas e vetores
- Percorre-se registro por registro em busca da chave

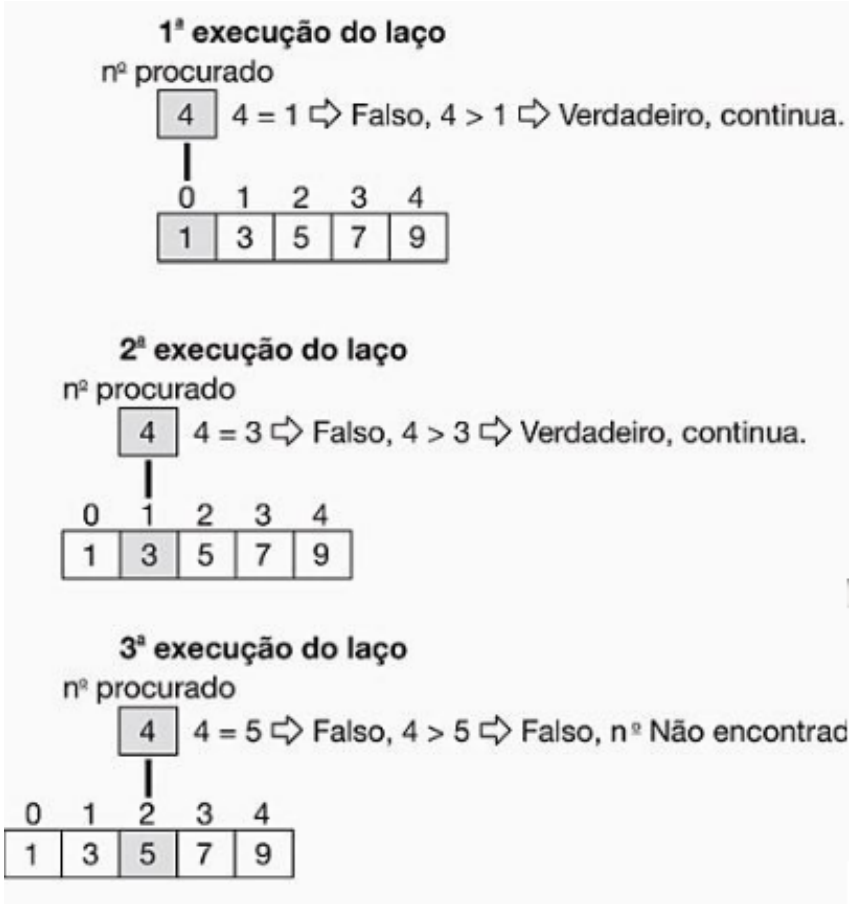
Comportamento - Vetor Não Ordenado



Algoritmo

```
UNORDERED-LINEAR-SEARCH[A, n, x]
for i ← 1 to n
        do if A[i] = x
                then return i
                else  i ← i + 1
return "x not found"
```

Comportamento - Vetor Ordenado



Algoritmo

```
ORDERED-LINEAR-SEARCH[A, n, x]
for i ← 1 to n
        do if A[i] = x
                then return i
            elseif A[i] < x
                then i ← i + 1
            else return "x not found"
return "x not found"
```

Análise Assintótica

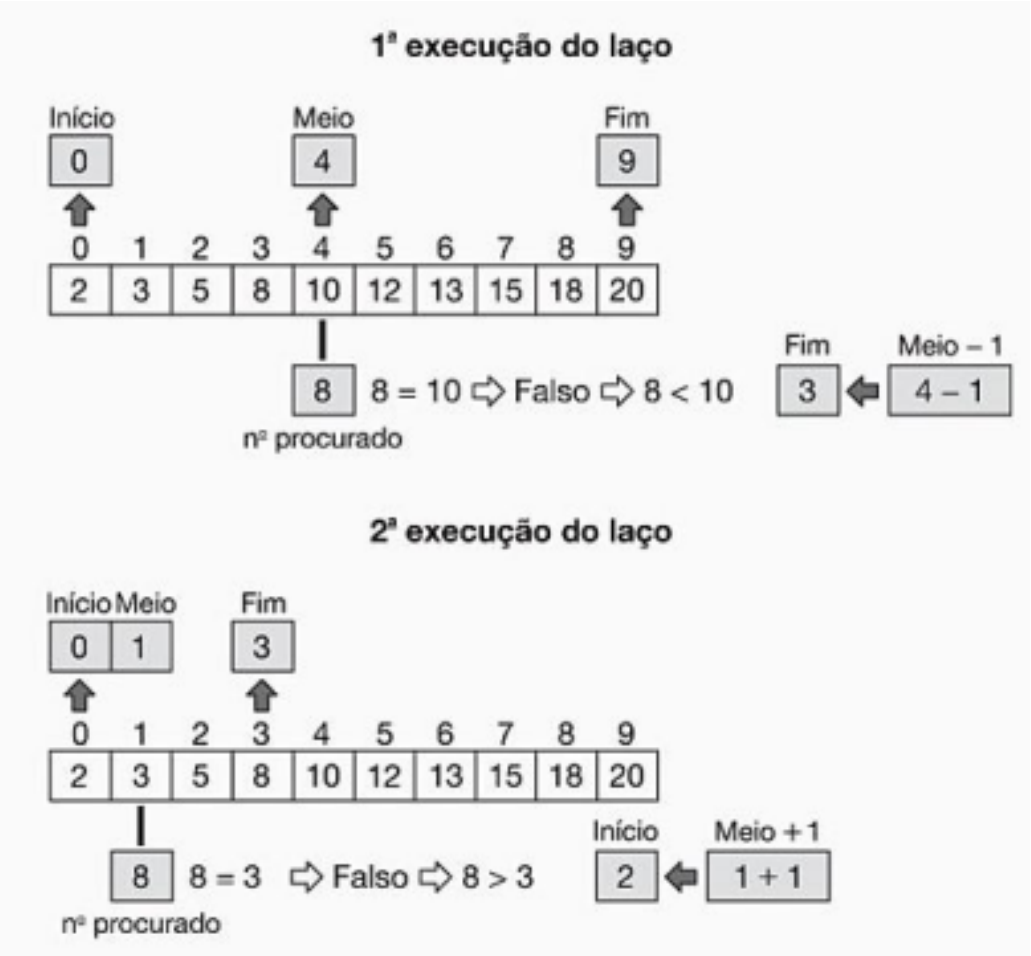
- Dado um vetor de tamanho n

  Pior Caso $O(n)$ Quando se encontra na última posição ou não é encontrado

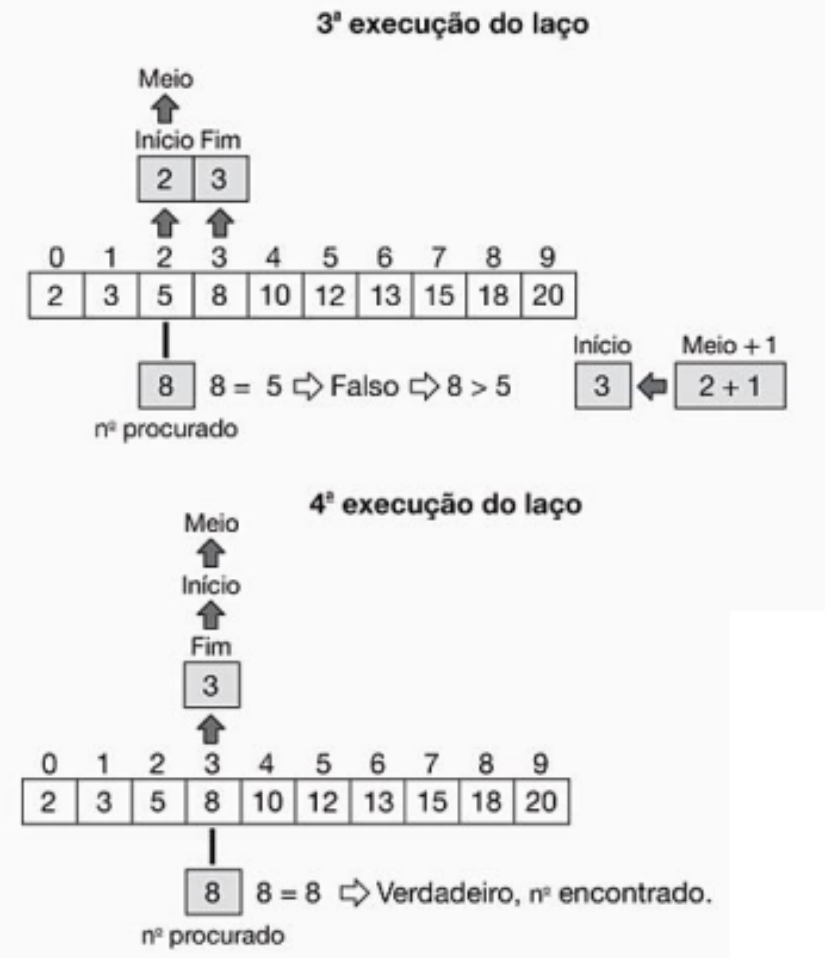
  Melhor caso $O(1)$

- É executado somente em dados ordenados
- O elemento buscado é comparado ao elemento do meio do arranjo
  - Se igual, busca bem-sucedida,
  - Se menor, busca-se na metade inferior do arranjo,
  - Se maior, busca-se na metade superior do arranjo.

Comportamento



Comportamento



Algoritmo

```
low ← 1
high ← n
while low ≤ high
        do mid ← ⌊(low + high)/2⌋
            if A[mid] = x
                then return mid
            elseif A[mid] < x
                then low ← mid + 1
            else high ← mid − 1
return "x not found"
```

Análise Assintótica

Pior Caso $O(\log n)$ pois cada comparação reduz o número de possíveis candidatos por um fator de 2.

Análise Geral

- Vantagens
  - Eficiência da busca
  - Simplicidade da implementação
- Desvantagens
  - Exige o uso de uma estrutura ordenada
  - Exige uso de índices **inteiros consecutivos**
  - Inserção e remoção de elementos são ineficientes
    - Realocação de elementos

- Faça um programa que cadastre 100.000 números aleatórios **não ordenados e sem repetição**. Calcule o tempo de execução (em segundos) para encontrar uma chave durante a mesma execução usando:
  - Busca sequencial
  - Busca Binária (Faça a ordenação)
  - Busca por Interpolação (Fazer pesquisa)

```
procedimento buscalinear(chave)
{}
procedimento buscaBinaria(chave)
{}
procedimento buscaInterpolacao(chave)
{}

main
{
    declara dadosAleatorios;
    declara chave;

    tempo_inicial = obtemTempo()
    buscalinear(chave);
    tempo_final = obtemTempo()
    tempo_execucao = tempo_final - tempo_inicial
    printa("Tempo de Execucao %s ",tempo_execucao)

}
```

Referências I

- Lee K.D., Hubbard S. (2015) Trees. In: Data Structures and Algorithms with Python. Undergraduate Topics in Computer Science. Springer, Cham. Retrieved from https://doi.org/10.1007/978-3-319-13072-9_6
- Hubbard, J. (2007). Schaum's Outline sof Data Structures with Java. Retrieved from http://www.amazon.com/Schaums-Outline-Data-Structures-Java/dp/0071476989
- Cormen, T. H., Leiserson, C. E., & Stein, R. L. R. E. C. (2012). Algoritmos: teoria e prática. Retrieved from https://books.google.com.br/books?id=6iA4LgEACAAJ.

Referências II

- Ascenio, Ana Fernanda Gomes. Estrutura de dados: Algoritmos, análise da complexidade e implementações em Java e C++. São Paulo: Pearson Prentice Hall, 2010.
- Szwarcfiter, Jayme Luiz. Estruturas de dados e seus algoritmos / Jayme Luiz Szwarcfiter, Lilian Markenzon. 3.ed. [Reimpr.]. - Rio de Janeiro : LTC, 2015.
- Capítulo 20: Árvores — documentação Aprenda Computação com Python 3.0 2009.1. Retrieved from https://mange.ifrn.edu.br/python/aprenda-com-py3/capitulo_20.html

Busca

Professor: Elton Sarmanho[1]
E-mail: eltonss@ufpa.br

[1]Faculdade de Sistemas de Informação - UFPA/CUTINS

3 de março de 2023